
# Database After Graduation

# Week 3

**By Narges Yarahmadi**
**Feb 2026**

## Table of Contents

# 1 Pagination strategies.

This is not a UI concern. This is a database performance concern disguised as a UI feature.

Let's begin from the core problem.

Imagine an Uber-like application has 5 million ride records.

Now a user opens their ride history page.

Should your API return 5 million rows?

Of course not.

Even returning 50,000 rows is dangerous:

- Large memory usage
- Slow database response
- Network congestion
- Slow frontend rendering
- Possible timeouts

So we never return "everything."

We return a **page**.

Pagination is the strategy of dividing large result sets into smaller, controlled chunks.

Now let's understand the purpose clearly.

Pagination solves four major problems:

1. Performance
2. Memory usage
3. Network efficiency
4. User experience

If pagination is wrong, your system will degrade under growth.

There are two main industrial strategies:

1. Offset-based pagination
2. Cursor-based pagination

Let's examine both carefully.

## 1.1 Offset-based Pagination

This is the most common and easiest to understand.

SQL example:

```
SELECT *
FROM rides
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;
```

This means:
Skip the first 40 rows.
Return the next 20 rows.

If page size = 20:
Page 1 → OFFSET 0
Page 2 → OFFSET 20
Page 3 → OFFSET 40

Simple.

In API form:

```
GET /rides?page=3&limit=20
```

Advantages:

- Easy to implement
- Easy to understand
- Works well for small datasets

But now we go deeper.

What happens if OFFSET = 1,000,000?

The database still scans through the first 1,000,000 rows internally before discarding them.

It does not “jump.” It walks.

That means:

- CPU waste
- Disk reads
- Slow queries at high offsets

Second problem: inconsistency.

Imagine:

User loads page 1.
Meanwhile, new rides are inserted.
User loads page 2.

Because the dataset changed, some records may:

- Appear twice
- Be skipped

Offset pagination is unstable in frequently changing datasets.

So offset is simple, but not always scalable.

---

## 1.2 Cursor-based Pagination

Now we move to industrial-grade design.

Instead of saying: “Give me page 3”

We say: “Give me records after this specific value.”

Example:

```
SELECT *
FROM rides
WHERE created_at < '2025-02-01 10:00:00'
ORDER BY created_at DESC
LIMIT 20;
```

Here, the timestamp acts as a cursor.

API example:

```
GET /rides?cursor=2025-02-01T10:00:00&limit=20
```

Instead of OFFSET, we use a position marker.

Advantages:

1. Much faster at scale
   Because it uses indexed comparisons instead of skipping rows.

2. Stable under inserts
   New rows do not disturb already fetched pages.
3. Ideal for infinite scroll systems

This is how:

- Twitter timelines
- Instagram feeds
- Ride history in large apps
  are implemented.

---

## 1.3 Key Design Requirements

Cursor-based pagination requires:

1. A stable ordering column
2. That column must be indexed
3. Ideally unique or combined with unique ID

Example safer pattern:

```
WHERE (created_at, id) < (?, ?)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

This avoids duplicate ordering when timestamps are identical.

---

## 1.4 When to Use Which?

Use offset-based when:

- Dataset is small
- Admin dashboards
- Internal tools
- Page numbers are required

Use cursor-based when:

- Dataset is large
- Data changes frequently
- Infinite scroll UI
- High-traffic systems

## 1.5 Advanced Industrial Concerns

### 1.5.1 Total Count

Offset pagination easily allows:

```
SELECT COUNT(*) FROM rides;
```

Cursor-based pagination often avoids expensive counts.
Instead, systems show:
“Showing more results…”
without telling total pages.

### 1.5.2 Indexing

If you paginate by created_at but no index exists,
performance collapses.

Always paginate on indexed columns.

### 1.5.3 Compound Filtering

Real systems combine pagination with filters:

```
WHERE user_id = 42
AND status = 'COMPLETED'
AND created_at < ?
```

Indexes must support filtering + ordering.

### 1.5.4 Security

Never trust client-provided limit blindly.
Always enforce a maximum:

limit = min(requested_limit, 100)

Otherwise, someone requests:
limit=1000000

And your system suffers.

## 1.6 Mental Model

Pagination is not about dividing pages.

It is about controlling database workload.

Offset pagination says:
“Skip and return.”

Cursor pagination says:
“Continue from here.”

Offset is counting.
Cursor is positioning.

Industrial systems prefer positioning.

---

## 1.7 Final Conceptual Insight

As data grows, naive pagination becomes a scalability bottleneck.

The database does not struggle because of complex queries.
It struggles because of inefficient traversal.

Good pagination design is invisible when done correctly.
Bad pagination design appears as:

- Slow endpoints
- Increasing latency with page number
- High CPU usage on DB
- User complaints about missing data

Pagination is a performance contract between your API and your database.

# 2 Databases in Backend Architectures

Today we stop thinking about databases as tables and start thinking about them as part of a backend system.

In modern applications, users do not talk directly to databases. They talk to APIs. The API talks to the database.

The real question is “How does data move safely and efficiently through an API?”

---

## 2.1 Where the Database Lives in Architecture

In a typical backend architecture, we have:

Client → API Server → Database

The client can be:

- Web browser
- Mobile app
- Another backend service

The API server:

- Receives HTTP requests
- Validates input
- Executes business logic
- Queries the database
- Returns JSON response

The database:

- Stores persistent data
- Executes queries
- Ensains consistency

Important idea:
The API is stateless.
The database is stateful.

Stateless means the API does not remember previous requests.
Stateful means the database stores long-term data.

This separation is fundamental in production systems.

## 2.2 API Request Lifecycle (One Request from Start to End)

Let us follow one request.

Example:
GET /users/42

Here is what happens internally:

1. Client sends HTTP request.
2. FastAPI receives the request.
3. Router matches the path.
4. Dependency injection creates a database session.
5. Business logic executes.
6. SQL query runs.
7. Database returns rows.
8. Data is serialized to JSON.
9. Response is sent to client.

Important understanding:
Each request creates work for the database.
Many requests mean many queries.
The database can become the bottleneck.

We will look into Life cycle deeper in next section.

# 3 API Request Lifecycle

When we talk about an “API request lifecycle,” we are describing the complete journey of a single request from the moment a client decides to ask for something until it receives a response. That’s it. A request is just a question. The lifecycle is the story of how that question gets answered.

We start with the core concept.

An API is simply a contract. API stands for Application Programming Interface. It defines how one software system talks to another. In web systems, APIs usually communicate using HTTP. HTTP is a text-based protocol that defines how requests and responses are structured.

An HTTP request has four main parts:

- Method such as GET, POST, PUT, DELETE
- Path such as /users/42
- Headers such as Content-Type
- Optional body, usually JSON

An HTTP response has:

- Status code such as 200 or 404
- Headers
- Body

That is the outer layer.

Now let’s move inside.

We imagine a real system:
Client → FastAPI backend → MySQL database.

## 3.1 Step 1: The client sends a request

A browser, mobile app, or another backend sends:

GET /users/42

This request travels over the network using TCP. TCP guarantees reliable delivery. HTTP sits on top of TCP and gives structure to the message.

## 3.2 Step 2: The server receives the request

Your FastAPI app does not directly listen to the network. Instead, an ASGI server such as Uvicorn listens for connections.

When a request arrives:

- The TCP connection is accepted.
- The raw HTTP text is parsed.
- The request is converted into a structured Python representation.

At this point, the server understands:
Method: GET
Path: /users/42
Headers
Body (if any)

Now the lifecycle begins inside your application.

## 3.3 Step 3: Routing

Routing answers a simple question:

Which function should handle this request?

In FastAPI, you define routes like this:

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
```

FastAPI checks:
Does the method match GET?
Does the path match /users/{user_id}?

If yes, it selects this function.
If not, it returns 404 immediately.

Routing is pattern matching. No business logic yet.

## 3.4 Step 4: Validation

Before your function runs, FastAPI validates inputs.

If user_id is declared as int and someone sends:

GET /users/abc

FastAPI rejects the request with a 422 error.
Your function is never executed.

If the request has a JSON body, FastAPI validates it against a Pydantic model. That means:

- Required fields must exist.
- Types must match.
- Formats can be enforced.

This step protects the system. In production systems, input validation is one of the strongest defensive mechanisms against bugs and attacks.

## 3.5 Step 5: Dependency resolution

Many endpoints need shared resources such as:

- Database connections
- Authentication info
- Configuration objects

FastAPI uses dependency injection to provide these.

Example:

```
def get_user(user_id: int, db: Session = Depends(get_db)):
```

Before executing your function, FastAPI calls get_db().

This is where connection pooling becomes important.

A database connection is expensive to create. Opening and closing a connection for every request would destroy performance. So ORMs like SQLAlchemy use a connection pool. A pool keeps a limited number of open connections and reuses them.

This means:
Request arrives → borrow connection from pool → execute query → return connection to pool.

Efficient. Scalable.

## 3.6 Step 6: Business logic

Now your function finally executes.

Example:

```
user = db.query(User).filter(User.id == user_id).first()
```

The ORM translates this Python expression into SQL:

```
SELECT * FROM users WHERE id = 42 LIMIT 1;
```

That SQL is sent to MySQL.

Inside MySQL:

- The query is parsed.
- A query plan is built.
- Indexes are used if available.
- Data is retrieved.
- Result rows are returned.

The ORM converts those rows into Python objects.

Now you have data.

## 3.7 Step 7: Serialization

Python objects cannot be sent over HTTP directly. They must be converted into JSON.

Serialization is the process of converting Python objects into JSON format.

FastAPI does this automatically using Pydantic models or response schemas.

Example output:

```
{
"id": 42,
"name": "Alice",
"email": "alice@example.com"
}
```

Serialization also filters fields. If your database model contains a password hash, it will not be included unless explicitly allowed. This is critical for security.

## 3.8 Step 8: Response construction

FastAPI builds an HTTP response:

- Status code 200
- Headers including Content-Type: application/json
- JSON body

This response is sent back through Uvicorn, over TCP, across the network, back to the client.

Lifecycle complete.

Now let’s zoom out and see the full architecture flow:

Client
→ Network (TCP + HTTP)
→ ASGI server
→ Router
→ Validation
→ Dependency injection
→ Business logic
→ Database
→ Serialization
→ HTTP response
→ Client

In industrial systems, there are additional layers.

### 3.8.1 Middleware

Middleware runs before and after your endpoint. It can:

- Log requests
- Measure execution time
- Handle authentication
- Enforce rate limits
- Add security headers

### 3.8.2 Authentication

Often a request carries a JWT token in headers. Middleware verifies it before the request reaches your endpoint. If authentication fails, the request stops there.

### 3.8.3 Error handling

If something crashes inside business logic, exception handlers convert Python exceptions into proper HTTP responses like 500 or 400.

### 3.8.4 Observability

Modern systems track:

- Request latency
- Error rates
- Database query time
- Throughput

Without observability, scaling is guesswork.

Now the conceptual insight.

An API request lifecycle is not just about “handling a request.” It is about controlled transformation:

Raw network bytes
→ structured request
→ validated input
→ domain logic
→ database interaction
→ structured output
→ network bytes again

It is a pipeline. Each stage has a responsibility. Clean backend design means responsibilities are separated:

Routing decides who handles the request.
Validation protects the system.
Business logic applies domain rules.
Data layer communicates with storage.
Serialization defines what the outside world sees.

When these responsibilities blur, systems become fragile.

When they are cleanly separated, systems scale.

And here is the subtle but powerful realization: every performance issue, every security issue, and every scaling problem in backend systems happens somewhere along this lifecycle. If students deeply understand this pipeline, they can diagnose real-world backend problems instead of just writing endpoints.

That is the API request lifecycle.

# 4 The Art of Conversation – Access Patterns & Pooling

When your API talks to a database, it is having a conversation. And like every conversation, there is a structure.

## 4.1 Part 1: What are Database Access Patterns?

**Database Access Patterns** are standardized ways that applications communicate with databases. They're like "conversation protocols" between your code and your data.

Real-World Analogy:
Think of a library:

- The library = Your Database
- The librarian = Database Connection
- You = Your Application
- The book = The Data

Different ways of asking for books = Different Access Patterns
Why Do Patterns Matter?
python

```
# Without pattern - Chaos!
def get_user_data(user_id):
    # Random way of accessing data
    connection = create_connection()  # Open
    cursor = connection.cursor()
    cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")
    result = cursor.fetchone()
    connection.close()  # Close
    return result

# With pattern - Organized!
def get_user_data(user_id):
    with get_db_session() as session:
        return session.query(User).get(user_id)
```

## 4.2 Part 2: The Three Core Patterns

### 4.2.1 Pattern 1: The "Open-Use-Close" Pattern (Basic)

Python

```
# 🏛️ Library Analogy:
# You go to library, ask for book, read it, leave

def basic_pattern():
    # 1. OPEN connection
    connection = create_connection()

    try:
        # 2. USE connection
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM products")
```

```
        results = cursor.fetchall()
        return results

    finally:
        # 3. CLOSE connection
        connection.close()

# ❌ Problem: Opening connection for EVERY request is expensive
# 🔧 Best for: Batch jobs, scripts, one-time operations
```

**Visual Representation:**

```
Request 1: [OPEN]──[USE]──[CLOSE]
Request 2:          [OPEN]──[USE]──[CLOSE]
Request 3:                    [OPEN]──[USE]──[CLOSE]

Time:    → → → → → → → → → → → → → → → → → →
```

### 4.2.2 Pattern 2: The Connection Pool Pattern (Industrial)

python

```
# 🏊 Library Analogy:
# Librarians stay at their desks. You just walk up and ask.

# 1. Setup pool at application startup
pool = ConnectionPool(
    min_connections=10,  # Always 10 librarians ready
    max_connections=20   # Can hire 10 more if busy
)

def pool_pattern():
    # 1. BORROW connection from pool (cheap!)
    connection = pool.get_connection()

    try:
        # 2. USE connection
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM products")
        return cursor.fetchall()

    finally:
        # 3. RETURN connection to pool (not close!)
        pool.return_connection(connection)

# ✅ Perfect for: Web applications, APIs, high-traffic systems
```

**Visual Representation:**

```
Pool: [CONN1][CONN2][CONN3]...[CONN10]

Request 1: ──BORROW CONN1──> [USE] ──RETURN CONN1──>
```

```
Request 2: ──BORROW CONN2──> [USE] ──RETURN CONN2──>
Request 3: ──BORROW CONN3──> [USE] ──RETURN CONN3──>

Time:    → → → → → → → → → → → → → → → → → →
```

### 4.2.3 Pattern 3: The Session Pattern (ORM Style)

python

```python
# 🗄️ Library Analogy:
# You get a library card (session) that works for your entire visit

from sqlalchemy.orm import sessionmaker

Session = sessionmaker(bind=engine)

def session_pattern():
    # 1. CREATE session (gets connection from pool)
    session = Session()

    try:
        # 2. USE session for multiple operations
        user = session.query(User).get(1)
        user.last_login = datetime.now()

        product = Product(name="Laptop", price=999)
        session.add(product)

        # 3. COMMIT all changes at once
        session.commit()

    except Exception:
        # 4. ROLLBACK on error
        session.rollback()
        raise

    finally:
        # 5. CLOSE session (returns connection to pool)
        session.close()

# ✅ Best for: Complex transactions, multiple related operations
```

---

## 4.3 Part 3: Real-World Implementation

### 4.3.1 The Industry Standard Pattern (FastAPI Style)

Python

```python
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from contextlib import contextmanager
```

```
# Step 1: Create Engine (The Pool Manager)
engine = create_engine(
    "mysql+pymysql://user:pass@localhost/db",
    pool_size=10,      # 10 permanent connections
    max_overflow=20,    # 20 temporary if needed
    pool_pre_ping=True  # Check connection health
)

# Step 2: Create Session Factory
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

# Step 3: Define Access Pattern
def get_db():
    """Dependency pattern - Each request gets its own session"""
    db = SessionLocal()
    try:
        yield db  # Session is provided to the endpoint
    finally:
        db.close()  # Always close! Returns connection to pool

# Step 4: Use in API
@app.get("/users/{user_id}")
def get_user(user_id: int, db: Session = Depends(get_db)):
    """Clean pattern: session automatically managed"""
    user = db.query(User).filter(User.id == user_id).first()
    return user
```

### 4.3.2 Pattern Comparison Table

| Pattern | Connection Management | Best For | Performance |
|---|---|---|---|
| **Open-Use-Close** | New connection each time | Batch jobs, scripts | ✗ Slow |
| **Connection Pool** | Reuse connections | Web apps, APIs | ✓ ✓ Fast |
| **Session Pattern** | Pool + transaction mgmt | Complex operations | ✓ Fast |

---

## 4.4 Part 4: Common Anti-Patterns (What NOT to do)

### 4.4.1 Anti-Pattern 1: The Connection Leak

python

```
# ✗ DANGEROUS - Connection never closed
def leak_pattern():
    connection = create_connection()
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM users")
    return cursor.fetchall()
```

```
    # Where's connection.close()? Memory leak!

# ✅ FIXED
def fixed_pattern():
    connection = create_connection()
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM users")
        return cursor.fetchall()
    finally:
        connection.close()  # Always closes
```

### 4.4.2 Anti-Pattern 2: The Death by a Thousand Connections

python

```
# ❌ DANGEROUS - New connection for every request
@app.get("/bad")
def bad_endpoint():
    # Each request creates NEW connection
    conn = mysql.connector.connect(
        host="localhost",
        user="root",
        password="password"
    )
    # ... database kills itself after 1000 connections

# ✅ FIXED - Use connection pool
@app.get("/good")
def good_endpoint(db: Session = Depends(get_db)):
    # Reuses connections from pool
    return db.query(User).all()
```

### 4.4.3 Anti-Pattern 3: The Transaction Turtle

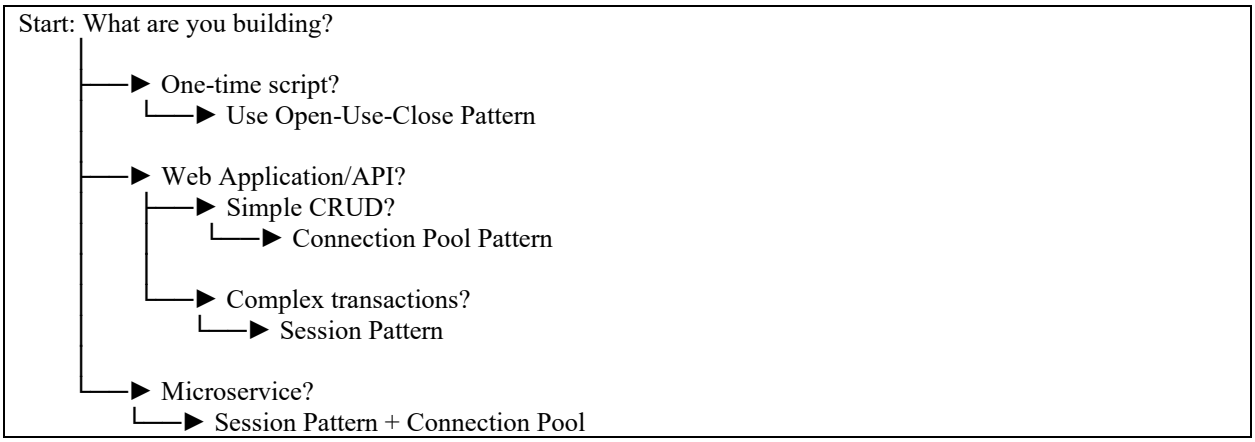
python

```
# ❌ SLOW - Committing in a loop
def slow_pattern(items):
    for item in items:
        session = Session()
        session.add(item)
        session.commit()  # COMMIT EVERY TIME - SLOW!
        session.close()

# ✅ FAST - Single commit
def fast_pattern(items):
    session = Session()
    try:
        for item in items:
            session.add(item)
        session.commit()  # ONE COMMIT - FAST!
    finally:
        session.close()
```

## 4.5 Part 5: Pattern Selection Guide

### 4.5.1 Decision Tree




### 4.5.2 Pattern Selection by Use Case

| Use Case | Pattern | Why |
|---|---|---|
| **Data Migration Script** | Open-Use-Close | Runs once, don't need pool |
| **High-Traffic API** | Connection Pool | Must handle many requests |
| **E-commerce Checkout** | Session Pattern | Multiple tables, need transaction |
| **Report Generator** | Open-Use-Close | Long-running, single task |
| **Real-time Dashboard** | Connection Pool | Many small, frequent queries |

## 4.6 Part 6: Live Demo - Patterns in Action

For a demo please refer to “demo_patterns.py” file on the BS under folder Week_3/demo_codes/.

## 4.7 Key Takeaways

### 4.7.1 Always Use Connection Pooling in Web Apps

python

```
# GOOD
engine = create_engine(url, pool_size=10)

# BAD
engine = create_engine(url)  # Default pool_size=5, but better than nothing!
```

### 4.7.2 One Session Per Request

python

```
# GOOD
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# BAD
db = SessionLocal()  # Global session - shared across requests!
```

### 4.7.3 Always Close Connections

Python

```
# GOOD - Context manager
with engine.connect() as conn:
    conn.execute(query)

# GOOD - Try/finally
conn = engine.connect()
try:
    conn.execute(query)
finally:
    conn.close()
```

### 4.7.4 Match Pattern to Use Case

- **Batch jobs**: Open-Use-Close
- **APIs**: Connection Pool
- **Complex transactions**: Session Pattern

---

# 5 Connection Pooling – The Party Line

Now we improve the design.

Instead of opening a new connection for every request, we create a **pool** of ready-to-use connections.

### 5.1.1 What is a connection pool?

It is a managed collection of open database connections that the application reuses.

### 5.1.2 How it works

- When the application starts, it creates a fixed number of connections. For example, 10.
- When a request arrives, it **borrows** one connection.
- It executes its query.
- It **returns** the connection to the pool.
- The connection is reused by the next request.

No repeated dialing. No repeated authentication. Just reuse.

Now the analogy changes.

Instead of making a new call for every question, imagine a conference call already in progress with 10 people on the line.
You just speak when you need to.
The hard part was setting up the call.
After that, it is efficient.

This is how real web applications survive traffic.

---

## 5.2 Industrial Context

Connection pooling is not optional in production systems.

### 5.2.1 1. Pool Size Limits

Every database has a maximum number of allowed connections.

For example:
MySQL → max_connections = 100

Now imagine:

- Your API pool size = 50
- You deploy 3 API instances

Total potential connections = 150

Your database only allows 100.
Result: connection failures. Possibly database crash.

This is why pool sizing must consider:

- Number of API instances

- Database max_connections
- Expected concurrency

Pooling improves efficiency, but misconfigured pooling can take down a system.

---

### 5.2.2 2. Tools in Python

In Python:

- **SQLAlchemy** has built-in connection pooling.
- FastAPI applications using SQLAlchemy automatically benefit from this.
- For high scale production systems, dedicated external poolers like **PgBouncer** sit between the app and the database to manage connections even more efficiently.

External poolers are useful when:

- You have many microservices
- You have many short-lived connections
- You need tighter control over database load

---

## 5.3 Key Takeaway

The database access pattern is always:

Open → Execute → Close

But in real systems, we do not physically open and close each time.

We borrow and return.

Connection pooling transforms inefficient conversations into sustained, controlled dialogue.

And in backend engineering, efficiency at this level is what separates a demo from a production system. We look into connection pool sizing in next section.

# 6 Connection Pool Sizing

It is one of the quiet forces that determines whether your API survives production traffic or collapses under it. Let’s begin from the core idea.

A database connection is not just a function call. It is a network socket, memory allocation, authentication handshake, session state, and server-side resource reservation. Opening a new connection for every request is expensive. Very expensive.

So we use a connection pool.

A connection pool is a managed collection of open database connections that can be reused. Instead of:

Request → open connection → query → close connection

we do:

Request → borrow connection → query → return connection

This reuse drastically reduces latency and CPU overhead.

Now here is the important part: the pool has a size.

And that size matters more than most people think.

Let's imagine your FastAPI service is deployed with 4 worker processes. Each worker can handle multiple requests concurrently. Suppose your connection pool size is 5 per worker.

That means your application can hold at most 20 active database connections.

Why is this important?

Because MySQL also has limits.

MySQL has a parameter called max_connections. If your application exceeds that number, new connections are rejected. In production, that means outages.

Now let's reason about sizing.

First principle:
A connection can execute only one query at a time.

If you have 50 concurrent requests and only 5 connections, 45 requests will wait. That waiting time increases latency.

So bigger pool = better performance?

Not necessarily.

Here is the twist.

Each database connection consumes:

- Memory on the DB server
- File descriptors
- CPU context switching overhead

If you open 500 connections but your database CPU has 8 cores, you are not magically 60x faster. You are just adding contention. The database will thrash between sessions.

This is where engineering thinking begins.

We need to balance three systems:

- Application concurrency
- Database capacity
- Query performance

Let's build a mental model.

Suppose:
Average query time = 50 milliseconds
You receive 200 requests per second
Each request performs one query

How many concurrent connections do you need?

Concurrency ≈ request_rate × query_time

So:
200 × 0.05 seconds = 10 concurrent queries

That means roughly 10 active connections are enough to sustain that load.

Not 100. Not 200. Around 10.

This simple formula is powerful. It turns guessing into reasoning.

Now let's connect this to FastAPI and SQLAlchemy.

In SQLAlchemy, you configure pool size like this:

```
engine = create_engine(
    DATABASE_URL,
    pool_size=10,
    max_overflow=20
)
```

pool_size is the number of permanent connections kept open.

max_overflow is how many extra temporary connections can be created when the pool is exhausted.

So if pool_size is 10 and max_overflow is 20, you could temporarily reach 30 connections.

This overflow is useful for traffic spikes. But it must be bounded. Unlimited overflow is dangerous.

Imagine you've built an app like Uber, with 100 users sending frequent ride requests. When these requests hit your backend, each one needs a connection to the MySQL database. But opening a connection every single time is costly. Instead, we use a connection pool: a set of ready-to-go connections. "Pool size" is simply how many of these connections you keep on standby. In that example, we estimate the number of concurrent database queries based on request rate and query time. If you have, say, 200 requests per second and each query takes 50 milliseconds, you'd need around 10 concurrent connections. That means your pool size should be roughly that number, so you can handle the load without delays. In short: pool size is how many ready connections you ke

## 6.1 Common Failure Modes

There are two common failure modes in production.

*First failure mode: pool too small*

Symptoms:

- Requests hang
- High latency
- Threads waiting for connections
- Application appears slow even though database CPU is low

What is happening?

Requests are waiting for a free connection. The bottleneck is not the database. It is the pool.

*Second failure mode: pool too large*

Symptoms:

- Database CPU spikes
- High context switching
- Query times increase
- MySQL shows too many active sessions

What is happening?

The database is overwhelmed by too many concurrent queries. The pool is flooding it.

The optimal pool size depends on:

- Number of application workers
- Average query time
- Database CPU cores
- Complexity of queries

Now a very important architectural detail.

If you use multiple application instances, each instance has its own pool.

For example:

3 containers
Each with pool_size = 15

Total potential connections = 45

You must multiply across instances. This is where many production systems fail. Developers size per instance and forget total capacity.

## 6.2 Async vs Sync

If your API is synchronous and blocking, each request thread may hold a connection while waiting for I/O. That increases required pool size.

If your API is asynchronous and releases the connection quickly after the query, you can often operate with a smaller pool.

Short transactions are good. Long transactions are dangerous. A transaction that holds a connection for 2 seconds blocks everyone else.

So best practice?

Keep database interactions short.
Avoid holding connections across external API calls.
Commit quickly.
Release early.

Now let's bring this into industrial best practices.

Start small.
Measure.
Adjust.

You do not guess. You observe.

Monitor:

- Database active connections
- Connection wait time
- Average query latency
- CPU usage on database

If connection wait time is high but DB CPU is low → increase pool.

If DB CPU is high and latency increases → reduce concurrency or optimize queries.

Scaling is a feedback loop, not a one-time configuration.

Here is the deeper philosophical insight.

A connection pool is a concurrency throttle.

It protects the database from being overwhelmed by the application.

In distributed systems, protection is more important than speed.

A well-sized pool prevents cascading failures.

Because when the database collapses, the entire system collapses.

Connection pool sizing is not about maximizing performance.
It is about controlling pressure.

And that is a very backend idea: engineering is not about going fast. It is about surviving load predictably.

# 7 ORM vs Raw SQL

**ORM** stands for **Object Relational Mapping**.

It is a technique that allows us to interact with a relational database using objects in our programming language instead of writing SQL queries manually.

In simple words:

Instead of writing:

```
SELECT * FROM users WHERE id = 5;
```

We write something like:

```
user = User.query.filter_by(id=5).first()
```

The ORM translates that Python code into SQL behind the scenes.

So ORM is a **layer between your application code and the database**.

It maps:

- Database tables → Classes
- Rows → Objects
- Columns → Attributes

That is why it is called Object Relational Mapping.

We are mapping objects to relational tables.

## 7.1 Why ORM Exists

Relational databases speak SQL.

Applications speak programming languages like Python, Java, or C#.

ORM acts like a translator between these two worlds.

| Without ORM: | With ORM: |
|---|---|
| • We write SQL strings manually<br>• We handle connections<br>• We map query results to objects manually | • We work with objects<br>• We let the framework generate SQL<br>• We reduce boilerplate code |

## 7.2 Example Without ORM (Raw SQL)

Let's see how it looks using raw SQL in Python:

```
cursor.execute("SELECT id, name, email FROM users WHERE id = %s", (user_id,))
row = cursor.fetchone()

user = {
    "id": row[0],
    "name": row[1],
    "email": row[2]
}
```

Notice:

- We manually write SQL
- We manually map the row to a dictionary
- We must handle SQL injection carefully
- We must manage connections

This gives us full control.

But it is more work.

## 7.3 Example With ORM

Now using ORM:

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String)
```

Then:

```
user = session.query(User).filter(User.id == user_id).first()
```

The ORM:

- Generates SQL automatically
- Converts result into a User object
- Handles escaping and safety
- Integrates with the rest of the application

Much cleaner.

## 7.4 ORM vs Raw SQL – The Trade-Offs

Now we move to the important part. This is not about which one is better. This is about **when to use which one**.

| Advantages of ORM | Disadvantages of ORM |
|---|---|
| 1.Developer Productivity<br>You write less code.<br>You think in objects instead of SQL syntax.<br>Faster development.<br>Very useful in:<br>Startups<br>MVPs<br>Rapid prototyping | 1. Performance Overhead<br>ORM generates SQL automatically.<br>Sometimes it generates inefficient queries.<br>For example:<br>N+1 query problems<br>Unnecessary joins<br>Over-fetching columns<br>If you don't understand SQL, ORM can hide performance issues. |
| 2. Maintainability<br>Schema changes are easier to manage.<br>ORM models centralize database structure.<br>Large teams benefit from this. | 2. Loss of Fine Control<br>With raw SQL, you control:<br>Index usage<br>Query hints<br>Execution plans<br>Complex joins<br>ORM can make advanced optimization harder. |
| 3. Security<br>ORM frameworks automatically:<br>Escape parameters<br>Reduce SQL injection risk<br>Safer by default. | 3. Learning Curve<br>People think ORM means "no need to learn SQL".<br>That is wrong.<br>To use ORM properly, you must understand:<br>Indexes<br>Joins<br>Query plans<br>Transactions<br>Otherwise, you create slow systems. |
| 4. Abstraction<br>You don't need to remember database-specific syntax.<br>Some ORMs are database-agnostic.<br>You can switch from MySQL to PostgreSQL more easily. | |

## 7.5 Advantages of Raw SQL

| | |
|---|---|
| Full Control | You write exactly the query you want.<br>You can optimize for:<br>• Performance<br>• Memory<br>• Index usage |
| Better for Complex Queries | For:<br>• Analytics<br>• Aggregations<br>• Window functions<br>• Complex reporting queries<br>Raw SQL is often cleaner and faster. |
| Predictable Performance | No hidden query generation.<br>What you write is what runs. |

## 7.6 So… Where Is ORM Used in Industry?

In real systems, most companies use **both**.

Typical pattern in backend APIs:

- CRUD operations → ORM
- Simple filtering → ORM
- Complex reporting → Raw SQL
- Performance-critical queries → Raw SQL

Large production systems often:

- Use ORM for 80 percent of operations
- Drop down to raw SQL for the heavy parts

This hybrid approach gives:

- Productivity
- Maintainability
- Performance

## 7.7 When Should You Use ORM?

| Use ORM when: | Do NOT rely only on ORM when: |
|---|---|
| • Building APIs<br>• Building admin dashboards<br>• Working with standard CRUD models<br>• Developing quickly<br>• Team is large and consistency matters | • Building analytics systems<br>• High-performance trading systems<br>• Very large-scale data processing<br>• Complex query tuning is required |

## 7.8 The Real Engineering Mindset

The key lesson for students:

ORM is a tool.

SQL is a language.

A backend engineer must understand both.

If you only know ORM, you are limited.
If you only write raw SQL everywhere, development becomes slow and harder to maintain.

The best engineers:

- Understand how SQL works internally
- Understand how ORM generates queries
- Know when to switch

## 7.9 Final Takeaway

- ORM increases productivity.
  Raw SQL increases control.
- Industry systems combine both.
- And most importantly:
- Using ORM does not remove the need to understand databases deeply.
- If you don't understand indexes, joins, and execution plans,
  ORM will not save you.

# 8 What is Swagger?

Swagger is an **interactive API documentation interface**.

FastAPI automatically generates it from your code.

So when you write:

```
@app.get("/products")
```

FastAPI:

- Registers the route
- Documents the route
- Builds UI for testing it

Swagger is like a control panel for your API.

You can:

- Click endpoints
- Fill parameters
- Click “Execute”
- See JSON response immediately

No Postman needed at first.

---

## 8.1 Try this step-by-step

After running uvicorn:

### 1. Step A

Open:

```
http://127.0.0.1:8000/health
```

You should see: {"status":"ok","db":1}

If yes → DB connection works.

---

2. Step B

Open Swagger:

```
http://127.0.0.1:8000/docs
```

You'll see:

- GET /health
- GET /products
- GET /customers/{customer_id}/orders
- GET /analytics/top-products

Click one.

Click **Try it out**.

Click **Execute**.

You'll see:

- Request URL
- Response body (JSON)
- Response code

That JSON is what your function returned.

---

## 8.2 What is happening behind the scenes?

When you call:

```
GET /products?category=phone&limit=5
```

FastAPI:

1. Parses parameters
2. Calls your Python function
3. SQLAlchemy queries MySQL
4. Converts results to dict
5. Returns JSON
6. Swagger displays it

It's literally your database talking through Python.